DOCUMENT DE RÉFÉRENCE TECHNIQUE

MarchéCI

Architecture microservices, espace commerçant et exploitation de la plateforme

VERSION	DATE	PLATEFORME
1.0	Novembre 2026	Docker Compose

SOMMAIRE

Structure du dossier

Ce dossier présente l'architecture, les choix d'implémentation et les procédures d'exploitation de MarchéCI. Il sert de référence pour l'installation, la maintenance et l'évolution de la plateforme.

01	Synthèse et périmètre	Positionnement, fonctions, indicateurs
02	Architecture logicielle	Services, flux, principes
03	Modèle de données	Schémas, relations, intégrité
04	API et contrats	Routes, authentification, conventions
05	Espace commerçant et photos	Parcours, téléversement, diffusion
06	Sécurité	Accès, secrets, réseaux, fichiers
07	Fiabilité et messagerie	Outbox, reprises, cohérence
08	Déploiement et exploitation	Compose, volumes, migrations
09	Supervision et performance	Zabbix, santé, charge
10	Validation	Tests, scénarios, critères
11	Évolutions et conclusion	Feuille de route technique

Convention de lecture. Les noms de services, de routes et de variables sont indiqués en police technique. Les montants sont exprimés en francs CFA entiers. Les dates et heures sont en UTC.

01 — SYNTHÈSE

Synthèse et périmètre

MarchéCI est une plateforme e-commerce conçue pour la coopérative des commerçants d'Adjamé, à Abidjan. Elle réunit une place de marché publique, un espace de gestion pour les commerçants et un back-office d'administration.

4	1	3	75
API MÉTIER	SPA REACT	RÔLES APPLICATIFS	TESTS UNITAIRES

Fonctions principales

DOMAINE	CAPACITÉS
Catalogue	Produits, catégories, prix, stock, recherche, filtres, photos et mise en avant.
Commandes	Panier navigateur, validation serveur des prix et du stock, instantanés de commande, suivi des statuts et annulation.
Comptes	Inscription, connexion, JWT, rôles `CLIENT`, `COMMERCANT` et `ADMIN`.
Espace commerçant	Création et gestion de produits, import de photos depuis la galerie ou l'appareil photo, choix de la couverture.
Notifications	Consommation des événements de commande, persistance, reprises et file de messages non traités.
Supervision	Sondes de santé, collecte Zabbix, alertes sur indisponibilité des services.

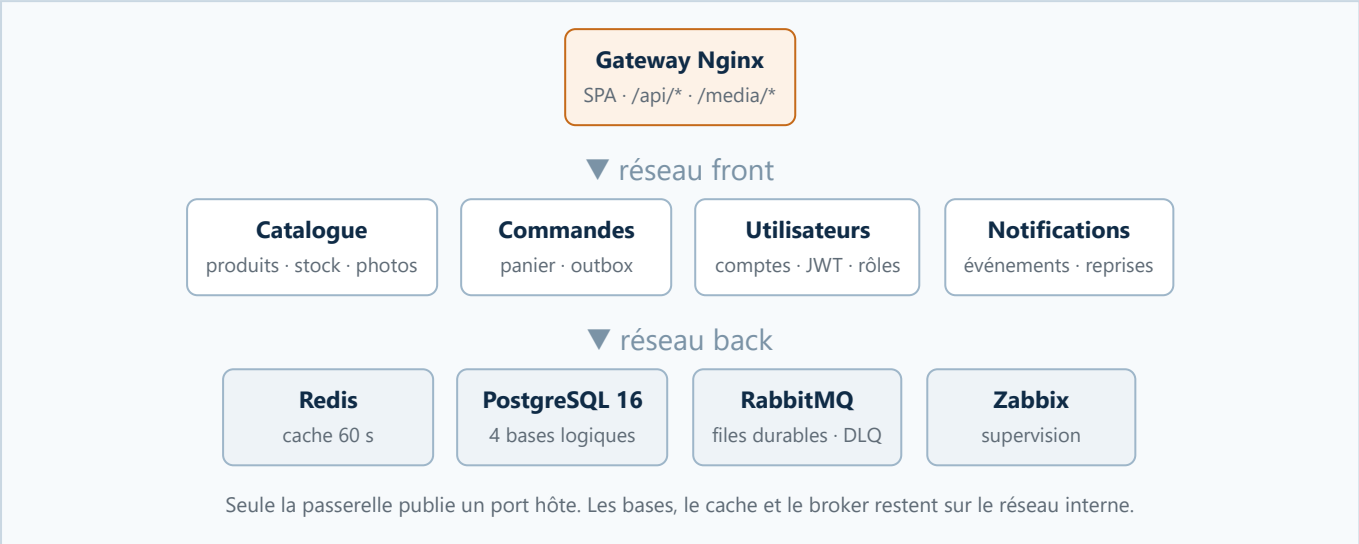
Hypothèses d'exploitation

- Déploiement sur un serveur Linux avec Docker Engine et Docker Compose v2.
- Un seul point d'entrée public : la passerelle Nginx, sur le port 80 par défaut.
- Quatre bases logiques PostgreSQL sur un serveur commun, avec des comptes SQL distincts.
- Les notifications e-mail et SMS sont persistées ; le branchement d'un fournisseur externe constitue une évolution.
- Le paiement en ligne est hors périmètre : la commande est réglée selon le parcours convenu avec le commerçant.

02 — ARCHITECTURE

Architecture logicielle

La plateforme applique une séparation par domaine métier. Chaque service possède sa base logique, expose une API HTTP et communique avec les autres par des contrats explicites.



Seule la passerelle publie un port hôte. Les bases, le cache et le broker restent sur le réseau interne.

Responsabilités

COMPOSANT	RESPONSABILITÉ	NE FAIT PAS
Catalogue	Produits, catégories, prix, stock, recherche, photos et couverture.	Ne gère ni comptes ni commandes.
Commandes	Validation des prix et du stock, instantanés, statuts, outbox.	Ne modifie pas les tables Catalogue.
Utilisateurs	Comptes, mots de passe, JWT, rôles et promotion des comptes.	Ne connaît ni produits ni commandes.
Notifications	Consommation RabbitMQ, persistance, reprises, DLQ.	Ne lit aucune base d'un autre service.
Gateway	Routage, distribution des répliques, fichiers média, timeouts.	Aucune logique métier.
Frontend	SPA React/TypeScript, panier local, formulaires et espace commerçant.	Ne calcule aucun prix faisant autorité.

Principes de conception

- **Services sans état** : les sessions sont portées par le JWT, ce qui permet la mise à l'échelle horizontale.
- **Base par service** : aucune lecture directe de la base d'un autre domaine.
- **Appels synchrones ciblés** : Commandes interroge Catalogue pour les prix et le stock ; le reste passe par événements.
- **Événements durables** : la commande et son événement sont écrits dans la même transaction PostgreSQL.
- **Point d'entrée unique** : la passerelle centralise le routage, les fichiers statiques et les limites de taille.

03 — DONNÉES

Modèle de données

Les données sont réparties par domaine. Les références entre services sont logiques : elles sont validées par les API et ne reposent pas sur des clés étrangères inter-bases.

Catalogue — `marcheci_catalogue`

TABLE	CHAMPS PRINCIPAUX	RÈGLES
`categories`	`id`, `slug`, `name`	Slug unique et indexé.
`products`	`id`, `slug`, `name`, `description`, `price_xof`, `category_id`, `stock`, `owner_id`, `image_url`, `is_active`, `created_at`, `updated_at`	Prix et stock positifs ; `owner_id` indexé ; soft-delete via `is_active`.
`product_images`	`id`, `product_id`, `filename`, `url`, `position`, `created_at`	FK vers `products.id` avec suppression en cascade ; couverture reprise dans `products.image_url`.

Index composite `(is_active, category_id)` pour les listes publiques et index sur `owner_id` pour l'espace commerçant.

Utilisateurs — `marcheci_utilisateurs`

TABLE	CHAMPS PRINCIPAUX	RÈGLES
`users`	`id`, `email`, `password_hash`, `full_name`, `phone`, `role`, `is_active`, `created_at`, `updated_at`	E-mail unique ; mot de passe haché en Argon2 ; rôles `CLIENT`, `COMMERCANT`, `ADMIN`.

L'inscription publique accepte `CLIENT` ou `COMMERCANT`. Le rôle `ADMIN` est créé au bootstrap et ne peut pas être attribué par l'API d'inscription.

Commandes — `marcheci_commandes`

TABLE	CHAMPS PRINCIPAUX	RÈGLES
`orders`	`id`, `user_id`, `status`, `total_xof`, `delivery_address`, `delivery_phone`, `payment_status`	Total recalculé côté serveur ; statuts de `EN_ATTENTE` à `LIVREE` ou `ANNULEE`.
`order_items`	`order_id`, `product_id`, `product_name`, `unit_price_xof`, `quantity`	Nom et prix copiés au moment de la commande.
`outbox_events`	`id`, `event_type`, `payload`, `published_at`, `attempts`, `last_error`	Écrit dans la même transaction que la commande ; publié par un worker.

Notifications — `marcheci_notifications`

TABLE	CHAMPS PRINCIPAUX	RÈGLES
`notifications`	`order_id`, `user_id`, `channel`, `recipient`, `template`, `payload`, `attempts`, `status`, `created_at`	`order_id` et `user_id` sont des références logiques ; statut `SIMULE_ENVOYE` ou `ECHEC`.

> **Intégrité.** Aucun service ne lit la base d'un autre. Les identifiants partagés sont des références logiques ; les instantanés de prix protègent l'historique des commandes des changements ultérieurs du catalogue.

04 — API

API et contrats

Les services exposent des API JSON documentées par OpenAPI. La passerelle unifie l'accès sous `/api/{service}/` et publie les documentations interactives sous `/docs/{service}/`.

Routage public

PRÉFIXE	SERVICE	USAGE
`/api/catalogue/`	Catalogue	Produits, catégories, photos, stock interne.
`/api/commandes/`	Commandes	Création, historique, détail, transitions.
`/api/utilisateurs/`	Utilisateurs	Inscription, connexion, profil, rôles.
`/api/notifications/`	Notifications	Historique administrateur, rejeu DLQ.
`/media/`	Gateway	Photos produits servies depuis le volume média.

Catalogue

MÉTHODE	ROUTE	ACCÈS ET RÉSULTAT
GET	`/products`	Public ; pagination, recherche, catégorie, disponibilité.
GET	`/products/{id}`	Public ; produit complet avec galerie ordonnée.
GET	`/products/mine`	Commerçant ou admin ; produits du propriétaire.
POST	`/products`	Commerçant ou admin ; crée le produit et renseigne `owner_id`.
PUT	`/products/{id}`	Admin ou propriétaire ; mise à jour partielle.
DELETE	`/products/{id}`	Admin ou propriétaire ; désactivation logique.
POST	`/products/{id}/images`	Admin ou propriétaire ; multipart, 8 Mo maximum, 10 photos.
DELETE	`/products/{id}/images/{image_id}`	Suppression et report de la couverture.
POST	`/products/{id}/images/{image_id}/cover`	Définit la photo de couverture.
POST	`/products/{id}/adjust-stock`	Appel interne authentifié ; ajustement de stock.

Commandes, comptes et notifications

SERVICE	ROUTES PRINCIPALES
Commandes	`POST /orders`, `GET /orders`, `GET /orders/{id}`, transition administrateur, annulation.
Utilisateurs	`POST /auth/register`, `POST /auth/login`, `GET/PUT /users/me`, `GET /users`, `PATCH /users/{id}/role`.
Notifications	`GET /notifications` (admin), `POST /internal/replay-dlq` (admin).

Conventions

- Corps JSON, dates ISO 8601 en UTC, montants en FCFA entiers.
- Authentification par `Authorization: Bearer <JWT>` sur les routes privées.
- Erreurs normalisées : `{"detail": "..."}` ; codes 401, 403, 404, 409, 413, 422 et 503.

Espace commerçant et gestion des photos

Le rôle `COMMERCANT` est géré par le service Utilisateurs. Un commerçant dispose d'un espace dédié pour créer ses produits et importer ses photos depuis un ordinateur ou un téléphone.

Parcours utilisateur

1. Le commerçant crée un compte depuis `/inscription` en choisissant « Vendre mes produits », ou un administrateur le promeut avec `PATCH /users/{id}/role`.
2. Après connexion, il est redirigé vers `/commercant` ; l'accès est réservé au rôle `COMMERCANT`.
3. Il crée un produit : nom, description, prix, catégorie, stock et photo de couverture. Le slug est généré automatiquement.
4. Il ajoute jusqu'à 10 photos par produit, choisit la couverture et peut supprimer une image.
5. Le produit apparaît dans la boutique publique s'il est actif et disponible.

Import des photos

Galerie

Sélection depuis l'appareil ou le téléphone via un champ fichier natif, avec aperçu immédiat.

Appareil photo

Prise de vue directe sur mobile grâce à l'attribut `capture`, ou caméra live dans le navigateur.

Glisser-déposer

Dépôt de plusieurs fichiers sur la zone d'import, avec liste de progression.

Gestion

Prévisualisation, suppression, choix de la couverture et remplacement sans quitter la page.

Traitement côté serveur

ÉTAPE	TRAITEMENT
Contrôle du fichier	Type autorisé (JPEG, PNG, WebP), taille maximale de 8 Mo, lecture du contenu par Pillow.
Redressement	Application de l'orientation EXIF pour éviter les photos pivotées.
Redimensionnement	Le côté le plus long est limité à 1 600 px, sans agrandissement.
Conversion	Enregistrement en JPEG avec un nom UUID dans `MEDIA_ROOT/products`.
Persistance	Ligne `product_images` avec URL publique `/media/products/<uuid>.jpg`.
Diffusion	Volume Docker `media_data` monté en lecture seule dans la passerelle, servie sous `/media/`.

Contrôle d'accès

- Un commerçant ne peut modifier ou supprimer que les produits dont `owner_id` correspond à son identifiant.
- `GET /products/mine` retourne ses produits, y compris ceux désactivés.
- Un administrateur conserve l'accès global et peut gérer les produits historiques sans propriétaire.
- Le rôle `ADMIN` est protégé : il ne peut pas être retiré via l'API de rôle.

06 — SÉCURITÉ

Sécurité des accès et des données

La sécurité repose sur une authentification centralisée, des rôles explicites, des secrets hors dépôt et une réduction de la surface exposée.

Authentification et autorisation

ÉLÉMENT	MISE EN ŒUVRE
Mots de passe	Hachage Argon2 ; le hash n'est jamais retourné par l'API.
Jeton	JWT signé en HS256, avec émetteur et audience validés par chaque service.
Rôles	`CLIENT` pour l'achat, `COMMERCANT` pour la vente, `ADMIN` pour l'administration.
Inscription	Seuls `CLIENT` et `COMMERCANT` sont acceptés ; `ADMIN` est créé au bootstrap.
Limitation	Compteur Redis sur les tentatives de connexion pour freiner les attaques par force brute.

Matrice d'accès

RESSOURCE	PUBLIC	CLIENT	COMMERÇANT	ADMIN
Liste et détail produits	Oui	Oui	Oui	Oui
Création de commande	—	Oui	Oui	Oui
Produits du commerçant	—	—	Oui	Oui
Écriture catalogue	—	—	Produits possédés	Tous
Photos produits	Lecture	Lecture	Produits possédés	Tous
Rôles et comptes	—	Profil	Profil	Gestion
Notifications et DLQ	—	—	—	Oui

Secrets et configuration

- `.env` contient les variables d'interpolation Compose et reste ignoré par Git.
- Les fichiers de `secrets/` sont générés localement et montés dans les conteneurs ; aucun secret n'est versionné.
- Les images d'infrastructure sont référencées par digest ; les mots de passe par défaut des composants sont remplacés.
- La passerelle tourne sous un UID non privilégié et écoute sur le port interne 8080.

Réseau et fichiers

- Seul le port de la passerelle est publié sur l'hôte ; PostgreSQL, Redis et RabbitMQ restent internes.
- Zabbix est lié à `127.0.0.1:8090` par défaut ; son interface n'est pas exposée publiquement.
- Les photos sont validées par Pillow, renommées en UUID et servies comme fichiers statiques ; aucun binaire n'est stocké en base.
- Les en-têtes et timeouts Nginx limitent les requêtes abusives ; la taille des téléversements est plafonnée à 12 Mo au niveau passerelle.

07 — FIABILITÉ

Fiabilité et messagerie

La cohérence entre la commande et son événement est assurée par un journal outbox. Les défaillances temporaires du broker, du cache ou d'un service ne provoquent pas de perte silencieuse.

Chaîne de commande



Si RabbitMQ est indisponible, l'événement reste dans l'outbox et le worker réessaie.

Mécanismes de reprise

MÉCANISME	COMPORTEMENT
Outbox transactionnel	La commande et `order.created` sont validés dans la même transaction PostgreSQL.
Files de reprise	Trois files à TTL croissant (2, 4 et 8 secondes par défaut) avant la file morte.
File morte	`order.created.dlq` durable ; rejeu administrateur limité à 100 messages par appel.
Idempotence	Notifications vérifie l'existence d'une notification pour `order_id` avant insertion.
Compensation stock	Si une étape échoue, une tentative de restauration du stock est déclenchée.
Cache	Redis est facultatif ; Catalogue continue de servir les requêtes si le cache est indisponible.

Comportement en cas d'incident

INCIDENT	EFFET OBSERVABLE	REPRISE
RabbitMQ indisponible	La commande reste enregistrée ; l'événement est en attente.	Publication automatique après retour du broker.
Catalogue indisponible	Commandes ne peut pas valider prix et stock.	Erreur explicite ; nouvelle tentative côté client.
Redis indisponible	Le cache est contourné ; la base est sollicitée.	Retour automatique au cache après rétablissement.
Notifications indisponibles	Les messages restent dans la file durable.	Consommation à la reprise du service.
Base indisponible	Le service concerné retourne un état non sain.	Redémarrage et vérification des migrations.

Contrôles de santé

Chaque service expose `/health`. Commandes et Notifications retournent HTTP 503 si une dépendance critique est indisponible ; Catalogue distingue la base et le cache. La passerelle expose également un contrôle de santé et les scripts de lancement vérifient les routes via le point d'entrée public.

08 — EXPLOITATION

Déploiement et exploitation

La plateforme se déploie avec Docker Compose. Les migrations Alembic et l'amorçage administrateur sont exécutés au démarrage, avant la mise à disposition des API.

Topologie Compose

ÉLÉMENT	RÔLE
`gateway`	Nginx : SPA, routage API, fichiers `/media/` .
`catalogue`	API produits, catégories, stock et photos ; 1 à 3 répliques.
`commandes`	API commandes, worker outbox et transitions.
`utilisateurs`	Comptes, authentification et rôles.
`notifications`	Consommateur RabbitMQ et historique.
`postgres` , `redis` , `rabbitmq`	Données, cache et messagerie.
`zabbix`	Pile de supervision séparée, activable.

Volumes persistants

VOLUME	CONTENU
`pgdata`	Données des quatre bases logiques PostgreSQL.
`rabbitdata`	Files durables et définitions RabbitMQ.
`media_data`	Photos produits ; écriture par Catalogue, lecture par la passerelle.
`zabbixdb`	Base de supervision, lorsque la pile Zabbix est activée.

Procédure de démarrage

1. Créer `.env` à partir de `.env.example` et remplacer les valeurs `CHANGE_ME` .
2. Générer les fichiers secrets locaux : `python scripts/sync_secrets.py` .
3. Lancer `docker compose up -d --build` ; les migrations et l'amorçage s'exécutent automatiquement.
4. Vérifier `docker compose ps` et l'accès à `http://localhost` .
5. Pour la supervision : `docker compose -f monitoring/zabbix/compose.yaml up -d` , puis le provisionnement.

Mise à l'échelle et sauvegardes

- Le Catalogue peut être lancé avec `--scale catalogue=3` ; Nginx répartit les requêtes via le DNS Docker.
- Les sauvegardes PostgreSQL doivent couvrir les quatre bases et être testées par une restauration périodique.
- Le volume `media_data` doit être sauvegardé avec les bases pour conserver la cohérence des URLs produits.
- Les définitions RabbitMQ peuvent être exportées pour reconstruire files, échanges et politiques.
- Les journaux des conteneurs et les contrôles `/health` constituent la première source de diagnostic.

09 — SUPERVISION

Supervision et performance

La supervision couvre la disponibilité de la passerelle, des quatre API et de l'agent Zabbix. Les campagnes de charge servent à dimensionner l'infrastructure et à valider les choix de cache.

Zabbix

ÉLÉMENT	CONFIGURATION
Pile	Serveur Zabbix, base PostgreSQL dédiée, interface web et agent.
Hôte	Groupe et hôte « MarcheCI » créés par API JSON-RPC.
Sondes HTTP	Gateway et quatre services ; valeur de type texte pour les réponses JSON.
Agent	`agent.ping` pour vérifier la présence de l'agent.
Déclencheurs	`nodata(63s)=1` avec priorité High sur chaque sonde.
Accès	Interface liée à `127.0.0.1:8090` par défaut.

Campagnes de charge

Les scénarios k6 sont conservés dans `tests/load/`. Le profil de référence combine 70 % de lectures répétées, servies par le cache Redis, et 30 % de recherches uniques, servies par PostgreSQL. Les campagnes s'exécutent sur un projet Compose isolé pour ne pas impacter une pile partagée.

PARAMÈTRE	VALEUR DE RÉFÉRENCE
Utilisateurs virtuels	50
Montée en charge	20 secondes
Palier	40 secondes
Répartition du trafic	70 % cache / 30 % base
Point d'entrée	Passerelle Nginx, comme le trafic client

Indicateurs à suivre

Disponibilité

État des sondes `/health`, incidents et résolutions dans Zabbix.

Latence

Percentiles p95 et p99 par chemin : cache, base et appels internes.

Saturation

CPU, mémoire, pool SQL, connexions RabbitMQ et espace disque des volumes.

Erreurs

Taux de réponses 4xx/5xx, messages en file morte et échecs de publication outbox.

Les résultats détaillés des campagnes sont archivés dans `docs/reports/`. Toute comparaison de répliques doit être réalisée sur un hôte dédié, avec les mêmes paramètres et les mêmes métriques.

Validation

La validation combine des tests unitaires, des tests d'intégration HTTP, des scénarios Docker isolés et des vérifications fonctionnelles de l'espace commerçant.

Couverture des tests

SUITE	VOLUME	OBJET
Catalogue	19 tests	Produits, catégories, cache, propriété, photos et couverture.
Utilisateurs	21 tests	Inscription, connexion, JWT, rôles et protection ADMIN.
Commandes	22 tests	Prix, stock, statuts, outbox et compensation.
Notifications	13 tests	Consommation, idempotence, reprises et DLQ.
Intégration HTTP	7 tests	Communication réelle entre services, sans RabbitMQ ni Redis.
Frontend	Build + lint	Compilation TypeScript, composants et qualité statique.

Scénarios fonctionnels validés

- Inscription et connexion d'un commerçant, redirection vers `/commercant`.
- Création d'un produit avec attribution correcte de `owner_id`.
- Téléversement d'une photo 2000 × 1500, conversion en JPEG 1600 × 1200 et diffusion sous `/media/`.
- Consultation publique du produit et de la galerie, puis accès à `/products/mine`.
- Suppression d'une photo et report automatique de la couverture.
- Parcours de commande avec contrôle du stock et notification persistée via RabbitMQ.
- Reprise après indisponibilité de RabbitMQ et rejeu d'un message en file morte.
- Collecte des sondes Zabbix et vérification de l'état des services.

Critères d'acceptation

CRITÈRE	RÉSULTAT
Les quatre suites unitaires passent	75 tests réussis.
Les tests d'intégration passent	7 tests réussis.
Le build frontend passe	`npm run build` réussi.
Le lint frontend passe	`npm run lint` sans avertissement.
La migration commerçant est appliquée	`products.owner_id` et `product_images` présents.
Le parcours commerçant fonctionne de bout en bout	Produit, photo, galerie et couverture vérifiés.

Reproductibilité. Les scénarios Docker utilisent un projet Compose dédié, avec ses propres réseaux, bases et volumes. Ils n'arrêtent pas les services de la pile principale.

Évolutions et conclusion

MarchéCI fournit un socle opérationnel pour la vente en ligne de la coopérative. L'architecture sépare clairement les domaines et permet d'ajouter des capacités sans remettre en cause les services existants.

Feuille de route technique

AXE	ÉVOLUTION
Notifications	Brancher un fournisseur e-mail/SMS, gérer les accusés de remise et les modèles localisés.
Paiement	Intégrer un prestataire mobile money ou carte, avec webhooks et rapprochement des commandes.
Comptes	Ajouter la réinitialisation de mot de passe et, si nécessaire, la vérification en deux étapes.
Observabilité	Centraliser les journaux, exposer des métriques applicatives et tracer les appels entre services.
Exploitation	Automatiser les sauvegardes, tester les restaurations et documenter les procédures d'incident.
Sécurité	Terminer TLS au niveau de la passerelle, renforcer la politique CSP et évaluer une session HttpOnly.
Catalogue	Ajouter la gestion des variantes, des promotions et des stocks par point de vente.
Analytique	Produire des tableaux de bord de ventes et de performance par commerçant.

Conclusion

La plateforme couvre le parcours complet de la boutique : découverte des produits, constitution du panier, validation serveur des prix et du stock, création de commande, notification et suivi. L'espace commerçant ajoute la gestion autonome des produits et des photos, avec un contrôle d'accès par propriétaire.

Le socle technique est prêt pour un pilote : déploiement Compose reproductible, migrations versionnées, secrets hors dépôt, supervision et scénarios de reprise. Les évolutions de la feuille de route permettent d'étendre progressivement le service sans modifier les principes d'architecture retenus.

> **Références documentaires.** Architecture détaillée : `docs/architecture/01-architecture-technique.md` . Modèles : `docs/architecture/02-modeles-de-donnees.md` . Réseaux : `docs/architecture/04-reseaux-et-communications.md` . Contrats API : `docs/api/specifications-api.md` .